

Rising Waters: A Machine Learning Approach to Flood Prediction

Project Documentation

Submitted By:

Tavisala Varshitha Reddy (Team Lead)

Tamadapalli Chandu

Desetty Revathi

P Rahul

Shaik Sameer Buddubhai Gari

G.Pullaiah College Of Engineering And Technology

Submitted To:

Skill Wallet

Project Domain:

Machine Learning using Python, Flask and IBM Cloud

Academic Year:

2025-2026

Acknowledgement

I express my sincere gratitude to Skill Wallet for providing the opportunity to work on the project titled "Rising Waters: A Machine Learning Approach to Flood Prediction." This project helped me gain practical knowledge in Machine Learning, Data Analysis, Data Preprocessing, Model Building, Flask Web Development, and GitHub.

I also thank my mentors and faculty members for their valuable guidance and continuous support throughout the project. Their suggestions and encouragement helped me complete every phase successfully.

Finally, I would like to thank my family and friends for their motivation and support during the completion of this project.

Abstract

Floods are among the most devastating natural disasters, causing loss of life, damage to infrastructure, and economic disruption every year.

Accurate and timely flood prediction plays an important role in disaster preparedness and emergency response.

This project, "Rising Waters: A Machine Learning Approach to Flood Prediction," aims to predict the possibility of flood occurrence using historical weather and rainfall data. The project uses Machine Learning classification algorithms including Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost to analyze meteorological parameters such as temperature, humidity, cloud cover, annual rainfall, and seasonal rainfall.

After comparing the performance of all models, the best-performing model is selected and integrated into a Flask-based web application. Users can enter weather values through a simple web interface and instantly receive a prediction indicating whether flood conditions are likely or not.

This system demonstrates how Machine Learning can support disaster management authorities by providing faster and more reliable flood risk predictions.

Introduction

Floods are one of the most common natural disasters affecting millions of people around the world. Heavy rainfall, overflowing rivers, poor drainage systems, and climate change increase the frequency and intensity of floods. Early prediction of floods allows authorities to take preventive measures such as evacuation planning, resource allocation, and emergency response.

Machine Learning has become an effective technology for analysing large datasets and identifying patterns that are difficult to detect using traditional methods. By training classification algorithms on historical weather data, it is possible to predict flood risk with improved accuracy.

In this project, a Machine Learning model is trained using weather-related features such as temperature, humidity, cloud cover, annual rainfall, and seasonal rainfall. The best-performing model is deployed using Flask to create a simple web application that provides flood predictions based on user input.

Problem Statement

Flood prediction using traditional methods is often time-consuming and less accurate due to the complexity of weather conditions. Delayed predictions reduce the time available for authorities to respond, increasing the risk of damage and loss of life. There is a need for an intelligent, automated system capable of providing faster and more accurate flood predictions.

Objectives

- Develop a Machine Learning model for flood prediction.
- Analyse historical weather and rainfall data.
- Compare multiple classification algorithms.
- Select the best-performing model.
- Deploy the trained model using Flask.
- Provide a user-friendly web interface.
- Support early warning and disaster preparedness.

Technologies Used

The Flood Prediction System is developed using Python and Machine Learning technologies. The project involves data analysis, model training, web application development, and deployment. The following technologies and libraries were used throughout the project.

Table:

Technology / Tool	Purpose
Python	Programming Language
Jupyter Notebook	Data Analysis and Model Development
Pandas	Data Manipulation
NumPy	Numerical Computation
Matplotlib	Data Visualization
Seaborn	Statistical Visualization
Scikit-learn	Machine Learning Algorithms
XGBoost	Advanced Classification Model
Flask	Web Application Framework
Joblib	Saving and Loading ML Models
HTML	Web Page Structure
CSS	User Interface Styling
Git	Version Control
GitHub	Source Code Repository
IBM Cloud	Cloud Deployment Platform

System Requirements

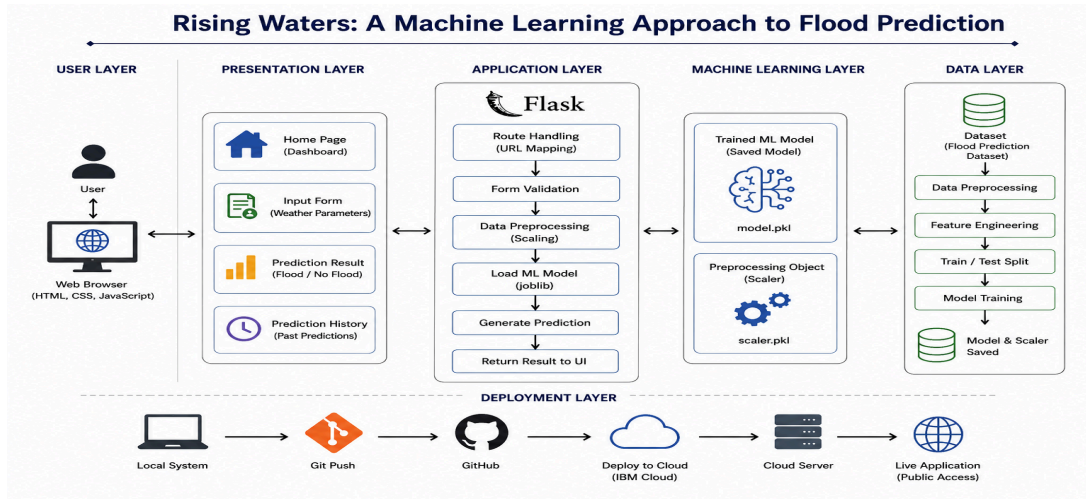
Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: 4 GB or above
- Storage: 10 GB Free Space
- Internet Connection

Software Requirements

Software	Version
Windows 10 / 11	Operating System
Python	3.10 or above
Anaconda Navigator	Latest
Jupyter Notebook	Latest
Flask	Latest
Git	Latest
GitHub Account	Required
Web Browser	Chrome / Edge

System Architecture



The system architecture consists of six major layers.

User Layer

The user accesses the application through a web browser and enters weather-related information.

Presentation Layer

The presentation layer contains the Home Page, Prediction Form, Prediction Result Page, and Prediction History.

Application Layer

The Flask application handles routing, input validation, preprocessing, model loading, and prediction generation.

Machine Learning Layer

The trained XGBoost model and scaler are loaded to predict flood risk.

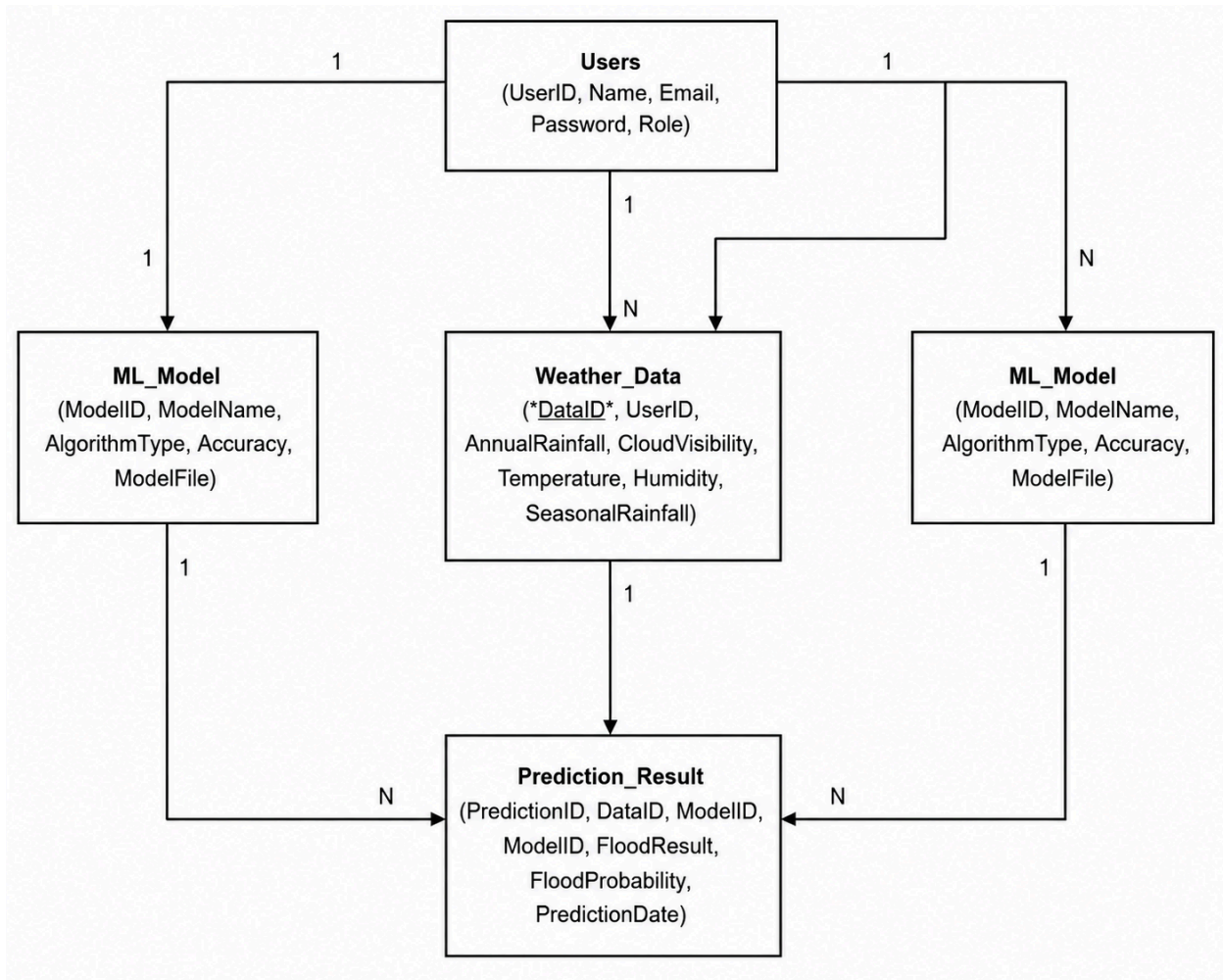
Data Layer

Historical flood datasets undergo preprocessing, feature engineering, train-test splitting, and model training.

Deployment Layer

The application is deployed using GitHub and IBM Cloud, making it accessible through the web.

Entity Relationship Diagram



The Entity Relationship Diagram represents the relationship between the User and Prediction entities. Each prediction is generated based on weather parameters entered by the user. The prediction result indicates whether flood conditions are likely or not.

Dataset Description

The flood prediction dataset used in this project was collected from Kaggle. It contains historical weather-related information and a target variable indicating flood occurrence.

The dataset includes the following features:

- Temperature
- Humidity
- Cloud Cover
- Annual Rainfall
- January–February Rainfall
- March–May Rainfall
- June–September Rainfall
- October–December Rainfall
- Average June Rainfall
- Sub Rainfall
- Flood (Target Variable)

Reading the Dataset

```
[26]: import os
      print(os.getcwd())
C:\Users\Welcome\Rising-Waters-Flood-Prediction\notebooks
```

```
[27]: import pandas as pd
      dataset = pd.read_excel("../dataset/flood.xlsx")
```

```
[28]: dataset.head()
```

```
[28]:
```

	Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood
0	29	70	30	3248.6	73.4	386.2	2122.8	666.1	274.866667	649.9	0
1	28	75	40	3326.6	9.3	275.7	2403.4	638.2	130.300000	256.4	1
2	28	75	42	3271.2	21.7	336.3	2343.0	570.1	186.200000	308.9	0
3	29	71	44	3129.7	26.7	339.4	2398.2	365.3	366.066667	862.5	0
4	31	74	40	2741.6	23.4	378.5	1881.5	458.1	283.400000	586.9	0

```
[31]: dataset.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 115 entries, 0 to 114
Data columns (total 11 columns):
 #   Column          Non-Null Count  Dtype  
---  --
 0   Temp            115 non-null   int64  
 1   Humidity        115 non-null   int64  
 2   Cloud Cover     115 non-null   int64  
 3   ANNUAL          115 non-null   float64 
 4   Jan-Feb         115 non-null   float64 
 5   Mar-May         115 non-null   float64 
 6   Jun-Sep         115 non-null   float64 
 7   Oct-Dec         115 non-null   float64 
 8   avgjune         115 non-null   float64 
 9   sub             115 non-null   float64 
10  flood           115 non-null   int64  
dtypes: float64(7), int64(4)
memory usage: 10.0 KB
```

Data Analysis

Before training the Machine Learning models, the dataset was analysed to understand feature distributions, identify patterns, and examine relationships among variables.

The following techniques were used:

- Descriptive Statistics
- Univariate Analysis
- Multivariate Analysis

Descriptive Analysis

```
42]: dataset.describe()
```

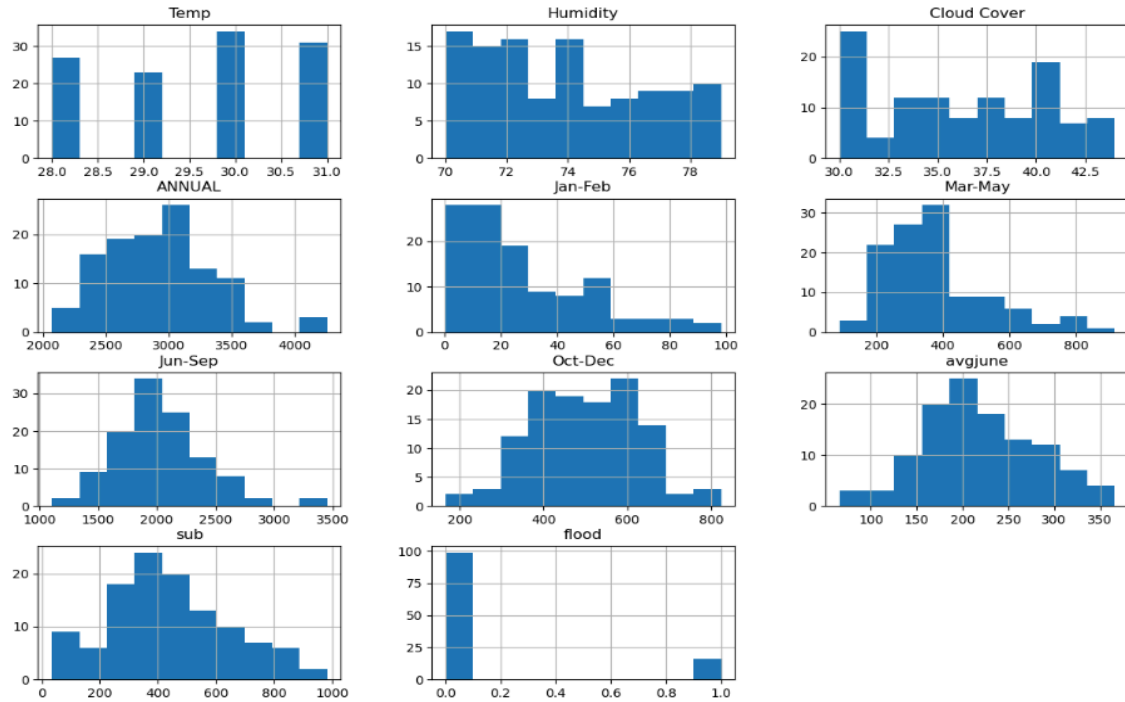
	Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood
count	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000
mean	29.600000	73.852174	36.286957	2925.487826	27.739130	377.253913	2022.840870	497.636522	218.100870	439.801739	0.139130
std	1.122341	2.947623	4.330158	422.112193	22.361032	151.091850	386.254397	129.860643	62.547597	210.438813	0.347597
min	28.000000	70.000000	30.000000	2068.800000	0.300000	89.900000	1104.300000	166.600000	65.600000	34.200000	0.000000
25%	29.000000	71.000000	32.500000	2627.900000	10.250000	276.750000	1768.850000	407.450000	179.666667	295.000000	0.000000
50%	30.000000	74.000000	36.000000	2937.500000	20.500000	342.000000	1948.700000	501.500000	211.033333	430.600000	0.000000
75%	31.000000	76.000000	40.000000	3164.100000	41.600000	442.300000	2242.900000	584.550000	263.833333	577.650000	0.000000
max	31.000000	79.000000	44.000000	4257.800000	98.100000	915.200000	3451.300000	823.300000	366.066667	982.700000	1.000000

Descriptive analysis provides statistical information such as mean, standard deviation, minimum, and maximum values.

Univariate Analysis

```
[36]: import matplotlib.pyplot as plt
import seaborn as sns
```

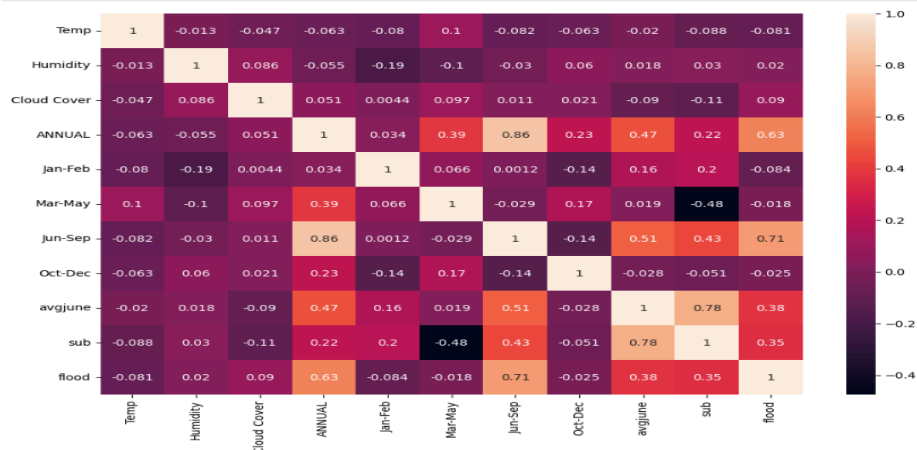
```
[37]: dataset.hist(figsize=(15,10))
plt.show()
```



Univariate analysis helps understand the distribution of individual features, while multivariate analysis reveals relationships between different variables using correlation analysis.

Multivariate Analysis

```
[41]: plt.figure(figsize=(12,8))
sns.heatmap(dataset.corr(), annot=True)
plt.show()
```



Data Preprocessing

Data preprocessing is one of the most important stages in Machine Learning. It prepares the dataset for training by cleaning and transforming the data.

The preprocessing steps performed in this project include:

- Handling Missing Values
- Handling Outliers
- Handling Categorical Values
- Train-Test Split
- Feature Scaling

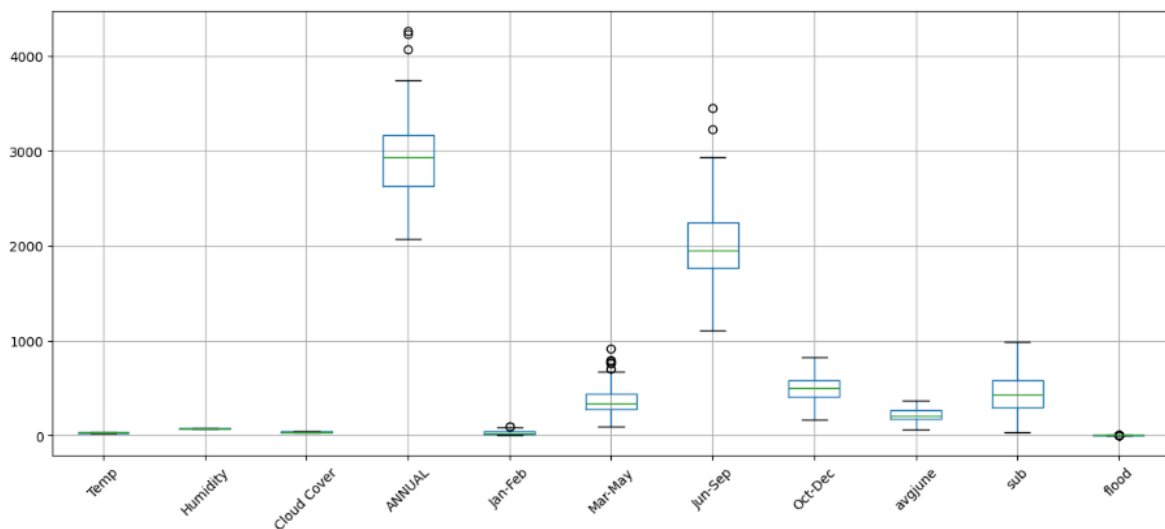
Handling Missing Values

```
[33]: dataset.isnull().sum()
```

```
[33]: Temp      0
      Humidity  0
      Cloud Cover  0
      ANNUAL    0
      Jan-Feb   0
      Mar-May   0
      Jun-Sep   0
      Oct-Dec   0
      avgjune   0
      sub       0
      flood     0
      dtype: int64
```

Handling Outliers

```
[44]: plt.figure(figsize=(15,6))
      dataset.boxplot()
      plt.xticks(rotation=45)
      plt.show()
```



Handling Categorical Values

```
[45]: dataset.dtypes
```

```
[45]: Temp          int64
      Humidity      int64
      Cloud Cover   int64
      ANNUAL        float64
      Jan-Feb       float64
      Mar-May       float64
      Jun-Sep       float64
      Oct-Dec       float64
      avgjune       float64
      sub          float64
      flood         int64
      dtype: object
```

Splitting Data into Training and Test Sets

```
[55]: X = dataset.drop('flood', axis=1)
      y = dataset['flood']
```

```
[58]: from sklearn.model_selection import train_test_split

      X_train, X_test, y_train, y_test = train_test_split(
          X, y,
          test_size=0.2,
          random_state=42
      )
```

```
[59]: print("Training Data Shape:", X_train.shape)
      print("Testing Data Shape:", X_test.shape)
```

```
Training Data Shape: (92, 10)
Testing Data Shape: (23, 10)
```

Feature Scaling

```
[60]: from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
[62]: print(X_train[:5])
```

```
[[-0.61039689 -0.66344859  0.15886699  1.08604492 -0.35983282  2.57623804
 -0.53467443  2.2628558  -1.6046566  -0.59664114]
 [-0.61039689  0.67069939 -0.8320322  1.06253895  0.45537729 -0.39714984
  1.68429033 -1.35274855  0.32161235  0.87878852]
 [ 1.23080028  0.67069939 -0.3365826  1.10384556  1.0579239  -0.38839723
  1.15383728  0.36324965  1.12702475  1.35588824]
 [ 0.3102017  1.33777339 -0.3365826  0.46713037 -0.35097184 -0.09455949
 -0.21844281  2.46118335  0.16497428  0.24379968]
 [-1.53099547 -1.33052258 -1.57520658  1.27249521 -0.09843392 -1.08610557
  2.21134304 -1.37134176  0.46307442  1.05580085]]
```

Model Building

After completing data preprocessing, the dataset was used to train multiple Machine Learning classification models. The objective was to identify the model that provides the highest prediction accuracy for flood occurrence.

The dataset was divided into training and testing sets using an 80:20 ratio. Four Machine Learning algorithms were implemented and evaluated.

Decision Tree

The Decision Tree classifier builds a tree-like structure by splitting the dataset based on feature values. It is simple to understand and suitable for classification problems.

Decision Tree Model

```
[63]: from sklearn.tree import DecisionTreeClassifier
      from sklearn.metrics import accuracy_score

[64]: dt_model = DecisionTreeClassifier(random_state=42)
      dt_model.fit(X_train, y_train)

[64]: ▾ DecisionTreeClassifier ⓘ ⓘ
      ▶ Parameters

[66]: dt_prediction = dt_model.predict(X_test)

[67]: dt_accuracy = accuracy_score(y_test, dt_prediction)
      print("Decision Tree Accuracy:", dt_accuracy)
      Decision Tree Accuracy: 1.0
```

Random Forest

Random Forest combines multiple decision trees to improve prediction accuracy and reduce overfitting. It generally performs better than a single decision tree.

Random Forest Model

```
[68]: from sklearn.ensemble import RandomForestClassifier
```

```
[69]: rf_model = RandomForestClassifier(  
        random_state=42  
    )  
  
    rf_model.fit(X_train, y_train)
```

```
[69]: ▾ RandomForestClassifier ⓘ ⓘ  
        ▶ Parameters
```

```
[70]: rf_prediction = rf_model.predict(X_test)
```

```
[71]: rf_accuracy = accuracy_score(  
        y_test,  
        rf_prediction  
    )  
  
    print("Random Forest Accuracy:", rf_accuracy)
```

Random Forest Accuracy: 1.0

K-Nearest Neighbors (KNN)

The KNN algorithm predicts the output by finding the nearest neighboring data points based on distance.

K-Nearest Neighbours Model

```
[72]: from sklearn.neighbors import KNeighborsClassifier
```

```
[73]: knn_model = KNeighborsClassifier(n_neighbors=5)  
  
    knn_model.fit(X_train, y_train)
```

```
[73]: ▾ KNeighborsClassifier ⓘ ⓘ  
        ▶ Parameters
```

```
[74]: knn_prediction = knn_model.predict(X_test)
```

```
[75]: knn_accuracy = accuracy_score(  
        y_test,  
        knn_prediction  
    )  
  
    print("KNN Accuracy:", knn_accuracy)
```

KNN Accuracy: 0.9130434782608695

XGBoost

XGBoost is an advanced gradient boosting algorithm that provides high prediction accuracy and efficient model training. It achieved the best performance among all the models used in this project.

XGBoost Model

```
[77]: from xgboost import XGBClassifier
```

```
[78]: xgb_model = XGBClassifier(  
      random_state=42,  
      eval_metric='logloss'  
      )  
  
      xgb_model.fit(X_train, y_train)
```

```
[78]: ▾ XGBClassifier ⓘ ⓘ  
      ▶ Parameters
```

```
[79]: xgb_prediction = xgb_model.predict(X_test)
```

```
[80]: xgb_accuracy = accuracy_score(  
      y_test,  
      xgb_prediction  
      )  
  
      print("XGBoost Accuracy:", xgb_accuracy)  
  
      XGBoost Accuracy: 1.0
```

Model Comparison and Evaluation

The performance of all Machine Learning models was compared using accuracy score, confusion matrix, and classification report. The comparison helped identify the most suitable model for deployment.

Model Comparison

Comparing the Models

```
[81]: print("Decision Tree :", dt_accuracy)
      print("Random Forest :", rf_accuracy)
      print("KNN :", knn_accuracy)
      print("XGBoost :", xgb_accuracy)

Decision Tree : 1.0
Random Forest : 1.0
KNN : 0.9130434782608695
XGBoost : 1.0
```

Among all the implemented algorithms, XGBoost achieved the highest prediction accuracy. Therefore, it was selected as the final model for deployment in the Flask web application.

Model Evaluation

Evaluating Performance

```
[82]: from sklearn.metrics import confusion_matrix
      from sklearn.metrics import classification_report
```

```
[83]: print(confusion_matrix(
      y_test,
      xgb_prediction
      ))
```

```
[[20  0]
 [ 0  3]]
```

```
[84]: print(classification_report(
      y_test,
      xgb_prediction
      ))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	20
1	1.00	1.00	1.00	3
accuracy			1.00	23
macro avg	1.00	1.00	1.00	23
weighted avg	1.00	1.00	1.00	23

The confusion matrix provides detailed information about correct and incorrect classifications, while the classification report includes precision, recall, F1-score,

and overall accuracy. These evaluation metrics confirm that the trained model performs effectively on unseen test data.

Saving the Model

```
import joblib

[85]: joblib.dump(
      xgb_model,
      "../model/floods.save"
      )

[85]: ['../model/floods.save']
```

The trained XGBoost model and feature scaler were saved using the Joblib library. These saved files are loaded by the Flask application during prediction.

Flask Web Application

A Flask web application was developed to provide an easy-to-use interface for flood prediction. Users enter weather-related parameters through a web form, and the trained Machine Learning model predicts whether flood conditions are likely.

The application consists of the following files:

- app.py
- index.html
- result.html
- flood.html
- noflood.html
- style.css

Home Page

 **Rising Waters**

Machine Learning Flood Prediction System

Temperature

Humidity

Cloud

Cover

Annual Rainfall

Jan-Feb

Rainfall

Mar-May Rainfall

Jun-Sep

Rainfall

Oct-Dec Rainfall

Average

June Rainfall

Sub

Rainfall

Predict Flood

The Home Page allows users to enter all required weather parameters for flood prediction.

Prediction Form



Rising Waters

Machine Learning Flood Prediction System

Temperature	30
Humidity	90
Cloud Cover	95
Annual Rainfall	1800
Jan-Feb Rainfall	120
Mar-May Rainfall	350
Jun-Sep Rainfall	1200
Oct-Dec Rainfall	450
Average June Rainfall	250
Sub Rainfall	180
Predict Flood	

The Prediction Form contains input fields for temperature, humidity, cloud cover, annual rainfall, seasonal rainfall, average June rainfall, and sub rainfall.



Flood Risk is Low

The Machine Learning model predicts that there is currently no significant chance of flooding.

The area appears safe based on the entered weather conditions.

[Predict Again](#)



High Flood Risk Detected

The Machine Learning model predicts that there is a high probability of flooding in this region.

Please take preventive measures immediately.

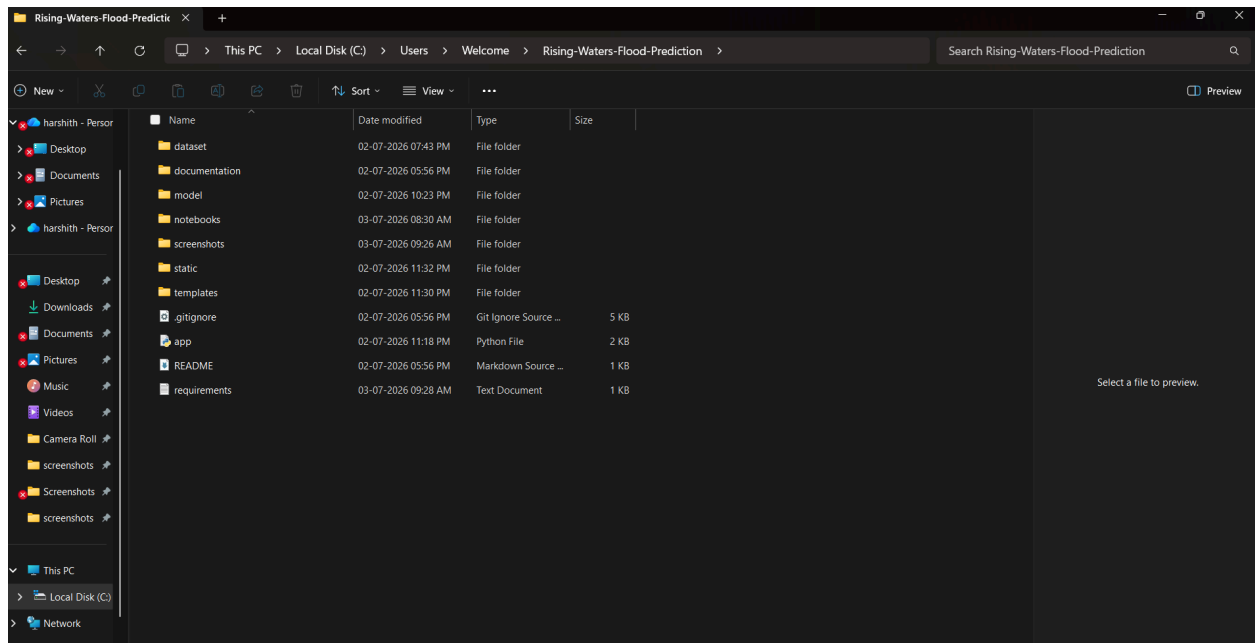
[Predict Again](#)

After entering the weather values, the application displays the prediction result indicating whether flood conditions are expected.

GitHub Repository

GitHub was used to maintain and organize all project files. The repository includes the source code, Machine Learning models, documentation, screenshots, datasets, HTML templates, CSS files, and notebooks.

The GitHub repository ensures proper version control and allows the project to be shared and accessed easily.



Results

The Flood Prediction System successfully predicts flood risk using Machine Learning algorithms. After comparing multiple classification models, XGBoost achieved the best overall performance and was selected as the final model.

The Flask web application provides an intuitive interface where users can enter weather data and receive immediate flood predictions. The application demonstrates how Machine Learning can support disaster preparedness and improve decision-making.

Future Scope

The project can be enhanced in several ways to improve prediction accuracy and usability:

- Integrate live weather data from APIs.
- Deploy the application on IBM Cloud or another cloud platform.
- Develop a mobile application for easy access.
- Include GPS-based regional flood prediction.
- Use Deep Learning techniques for improved performance.
- Send SMS or email alerts to users during high flood risk.
- Visualize flood-prone regions on an interactive map.

Conclusion

The project "**Rising Waters: A Machine Learning Approach to Flood Prediction**" successfully demonstrates the application of Machine Learning in disaster management.

Multiple classification algorithms were trained and evaluated using historical weather data. XGBoost achieved the best performance and was integrated into a Flask web application, allowing users to predict flood risk through a simple interface.

The project highlights the importance of Machine Learning in supporting early warning systems, improving emergency response, and helping authorities make informed decisions.

References

1. Kaggle – Flood Prediction Dataset
2. Python Documentation – <https://docs.python.org>
3. Pandas Documentation – <https://pandas.pydata.org>
4. NumPy Documentation – <https://numpy.org>
5. Scikit-learn Documentation – <https://scikit-learn.org>
6. XGBoost Documentation – <https://xgboost.readthedocs.io>
7. Flask Documentation – <https://flask.palletsprojects.com>
8. Matplotlib Documentation – <https://matplotlib.org>
9. Seaborn Documentation – <https://seaborn.pydata.org>
10. IBM Cloud Documentation – <https://cloud.ibm.com/docs>